

Samenvatting Toetsstof TP1 Informatica

Deel 1: Python

1.1 Variabelen, Geheugen en Datatypes

Het declareren van een variabele slaat een waarde op in het werkgeheugen (RAM). Python kent een intern referentie-adres toe en koppelt de variabelenaam daaraan. Wijs je een variabele een nieuwe waarde toe, dan verwijst de naam voortaan naar die nieuwe waarde.

Rollen van variabelen

- **Vaste waarde (constante):** Een waarde die na toewijzing ongewijzigd blijft gedurende het programma (in Python bij afspraak geschreven in **HOOFDLETTERS**, zoals `PI = 3.14159`).
- **Transformatie:** Een variabele waarvan de waarde ontstaat door een bewerking op een andere variabele.

```
lengte_m = 1.85
lengte_cm = lengte_m * 100 # Transformatie van lengte_m
```

- **Flag:** Een boolean variabele die een programmastatus bijhoudt.

```
game_over = False
ingelogd = True
```

- **Doelwaarde:** Een variabele die meeloopt in een lus om de beste match op te slaan (zoals het grootste getal, de kleinste waarde of een specifieke vondst).

```
scores = [12, 45, 78, 34]

# Start met het eerste element als beginpunt
hoogste = scores[0]
for score in scores:
    if score > hoogste:
        hoogste = score

# In Python kan dit ook direct:
hoogste = max(scores)
```

- **Counter:** Houdt bij hoe vaak een gebeurtenis plaatsvindt of hoe vaak een lus draait.

```
aantal_pogingen = 0
while aantal_pogingen < 3:
    aantal_pogingen += 1
```

- **Iterator Index:** Loopt stap voor stap door een lijst, reeks of tekst heen.

```
for letter in "Python":
    print(letter)  # 'letter' is de stapper
```

Regels en conventies voor variabelenamen

- Geen spaties toegestaan; gebruik underscores (`aantal_pogingen`).
- Moeten altijd beginnen met een letter of underscore (nooit met een cijfer).
- Mogen uitsluitend letters, cijfers en underscores bevatten.
- Grote getallen kun je leesbaar houden met underscores: `1_000_000` .
- Wetenschappelijke e-notatie werkt voor machten van 10: `5e9` staat voor 5×10^9 (resulteert in een `float`).

Invoer, Uitvoer en Omgeving

- `print()` : Schrijft gegevens naar het scherm. Meerdere argumenten scheid je met een komma.
- `#` : Commentaartekens. De Python-interpreter negeert alles achter dit teken op dezelfde regel.
- `type()` : Geeft het datatype van een variabele of waarde terug.
- **Basisdatatypes:**
 - `str` (String): Tekst tussen enkele (`'...'`) of dubbele (`"..."`) aanhalingstekens.
 - `int` (Integer): Een geheel getal (positief, negatief of nul).
 - `float` (Float): Een kommagetal, geschreven met een decimaalpunt (`.`).
 - `bool` (Boolean): Waarheidswaarde, uitsluitend `True` of `False` .

```
# Variabelen toewijzen en typen inspecteren
naam = "W Okan!"          # str
leeftijd = 43              # int
lengte = 1.85              # float
groot_getal = 5e9          # float (5 * 10^9)
is_student = False        # bool

print("Naam:", naam, "| Type:", type(naam))
print("Leeftijd:", leeftijd, "| Type:", type(leeftijd))
print("Groot getal:", groot_getal, "| Type:", type(groot_getal))
```

1.2 Invoer, Typeconversie (Casting) en Stringbewerking

De functie `input()` leest tekst in via het toetsenbord.

Belangrijk: `input()` levert **altijd** een `str` op, zelfs als de gebruiker een getal invoert.

Typeconversie (Casting)

Wil je met ingevoerde getallen rekenen, dan moet je de string expliciet omzetten: * `int()` :

Verandert een tekst of kommagetal naar een geheel getal (decimalen worden afgekapt). *

`float()` : Verandert tekst of geheel getal naar een kommagetal. * `str()` : Zet getallen of booleans om naar tekst.

```
# Typeconversie voorbeelden
tekst_getal = "42"
geheel_getal = int(tekst_getal)          # 42 (int)
kommagetal = float("3.14")              # 3.14 (float)
terug_naar_tekst = str(geheel_getal)    # "42" (str)
```

Tekstbewerking en f-strings

- `.upper()` : Zet alle letters om naar hoofdletters.
- `.lower()` : Zet alle letters om naar kleine letters.
- `.split()` : Splitst een tekst op in een lijst van woorden.
- `.replace("oud", "nieuw")` : Vervangt een stuk tekst door iets anders.
- **f-strings (formatted strings)**: Voeg variabelen en berekeningen direct in tekst in door een `f` voor het eerste aanhalingsteken te plaatsen en variabelen tussen `{...}` te zetten.

```
# Invoer omzetten van string naar getal
leeftijd_invoer = input("Voer je leeftijd in: ")
leeftijd = int(leeftijd_invoer)

# Tekst normaliseren met .lower() en .upper()
gebruikersnaam = input("Voer je naam in: ").lower()
kortingscode = input("Voer je kortingscode in: ").upper()

# f-string met berekening
getal1 = float(input("Eerste getal: "))
getal2 = float(input("Tweede getal: "))
print(f"Code {kortingscode} toegepast voor {gebruikersnaam}. Som: {getal1 + getal2}")

# Stringfuncties .split() en .replace()
zin = "Dit is een voorbeeld tekst."
woorden = zin.split()                                # ['Dit', 'is', 'een', 'voorbeeld', 'tekst.']
aangepast = zin.replace("voorbeeld", "nieuwe")        # "Dit is een nieuwe tekst."
```

1.3 Operatoren en Booleaanse Logica

Rekenkundige operatoren

Operator	Betekenis	Voorbeeld (<code>a = 10</code> , <code>b = 3</code>)	Resultaat
<code>+</code>	Optellen (of strings samenvoegen)	<code>a + b</code>	<code>13</code>
<code>-</code>	Aftrekken	<code>a - b</code>	<code>7</code>
<code>*</code>	Vermenigvuldigen	<code>a * b</code>	<code>30</code>
<code>/</code>	Normale deling (levert altijd <code>float</code>)	<code>a / b</code>	<code>3.3333...</code>
<code>//</code>	Gehele deling (rondt omlaag af, levert <code>int</code>)	<code>a // b</code>	<code>3</code>
<code>**</code>	Tot de macht	<code>a ** b</code> (10^3)	<code>1000</code>
<code>%</code>	Modulo (restwaarde na deling)	<code>a % b</code>	<code>1</code>

```
# toewijzen in het kort
x = 10
x += 5  # Zelfde als x = x + 5 (wordt 15)
x *= 2  # Zelfde als x = x * 2 (wordt 30)
```

Vergelijkingsoperatoren

Vergelijkingen leveren altijd `True` of `False` op: `*` `==` (gelijk aan) en `!=` (niet gelijk aan) `*` `>` (groter dan) en `<` (kleiner dan) `*` `>=` (groter dan of gelijk aan) en `<=` (kleiner dan of gelijk aan)

Logische operatoren

- `and` : `True` zodra **beide** voorwaarden waar zijn.
- `or` : `True` zodra **minstens één** voorwaarde waar is.
- `not` : Draait de waarheidswaarde om (`not True` → `False`).

Waarheidstabel

P	Q	P and Q	P or Q	not P
True	True	True	True	False
True	False	False	True	False
False	True	False	True	True
False	False	False	False	True

```
# Booleaanse logica in actie
leeftijd = 17
heeft_diploma = True

kan_starten = (leeftijd >= 16) and heeft_diploma
print("Toegelaten:", kan_starten) # True
```

1.4 Selecties: if, elif en else

Met een selectie bepaal je welke regels code uitgevoerd worden op basis van voorwaarden.

- **Indentatie:** Python gebruikt 4 spaties om codeblokken te groeperen.
- **Volgorde:** Python doorloopt de takken van boven naar beneden. Zodra een voorwaarde `True` oplevert, voert Python uitsluitend dat blok uit en slaat de rest over.

```
# Controle op schrikkeljaar en pariteit
jaar = int(input("Voer een jaartal in: "))

# Schrikkeljaar: deelbaar door 4, behalve bij eeuwwisselingen, tenzij
# deelbaar door 400
if (jaar % 4 == 0 and jaar % 100 != 0) or (jaar % 400 == 0):
    print(f"{jaar} is een schrikkeljaar.")
elif jaar % 2 == 0:
    print(f"{jaar} is geen schrikkeljaar, maar wel even.")
else:
    print(f"{jaar} is een oneven jaartal.")
```

1.5 Datastructuren: Lists

Een list is een geordende verzameling waarden die je na het aanmaken kunt wijzigen (munteerbaar).

Kernfuncties en Lijstbewerkingen

- **Indexering vanaf 0:** Het eerste element staat op index `0` (`lijst[0]`), het laatste op index `-1`.
- `len(lijst)` : Geeft het totale aantal elementen in de lijst terug.
- `.append(item)` : Voegt een element toe aan het einde van de lijst.
- `.insert(index, item)` : Voegt een element in op positie `index` ; de volgende elementen schuiven één plek op.
- `.pop(index)` : Verwijdert en retourneert het element op de opgegeven index (standaard de laatste).
- `.remove(waarde)` : Zoekt en verwijdert het eerste voorkomen van een specifieke waarde.
- **Slicing** (`lijst[start:stop]`) : Selecteert een deel van de lijst vanaf index `start` tot `stop` (exclusief).
- **Lidmaatschap** (`in`) : Controleert met `if item in lijst:` of een bepaalde waarde voorkomt.
- **Wiskundige bewerkingen:** `min(lijst)` , `max(lijst)` en `sum(lijst)` voor snelle berekeningen op getallenlijsten.

```
# Lijstbewerkingen in actie
fruit = ["appel", "banaan", "kers"]

# Toevoegen en invoegen
fruit.append("druif")           # Achteraan toevoegen
fruit.insert(1, "mango")        # Op index 1 invoegen

# Verwijderen met .pop() en .remove()
laatste = fruit.pop()           # Verwijdert "druif" en bewaart de waarde
fruit.remove("banaan")          # Verwijdert "banaan" op basis van de waarde

# Lengte en slicing
print("Aantal stuks fruit:", len(fruit)) # 3
print("Deel van de lijst:", fruit[0:2])  # ['appel', 'mango']

# Zoeken met 'in' en wiskundige functies
if "appel" in fruit:
    print("Appel gevonden!")

scores = [15, 42, 88, 23]
print(f"Som: {sum(scores)} | Laagste: {min(scores)} | Hoogste: {max(scores)}")
```

1.6 Datastructuren: Dictionaries

Een dictionary bewaart gegevens in sleutel-waardeparen (`key-value`). In plaats van een numerieke index vraag je waarden op via een unieke sleutel (vaak een string).

Eigenschappen

- Gemaakt met accolades (noem ze gewoon krulhaakjes 🤪) `{}` of `dict()` .
- Waarden toewijzen of opvragen via `woordenboek[sleutel]` .
- Met `.get(sleutel, standaardwaarde)` voorkom je een foutmelding (`KeyError`) als een sleutel niet bestaat.
- Waarden onder een sleutel kunnen complexe structuren zijn, zoals een list of een geneste dictionary.

```
# Dictionary aanmaken en uitbreiden
leerling = {
    "naam": "Aang",
    "klas": "A1A",
    "jaar": 2005
}

leerling["school"] = "Southern Air Temple"
leerling["hobbies"] = ["vliegen", "leren", "vechtsport"]

# opvragen met .get()
print("Naam:", leerling.get("naam"))
print("Cijfer:", leerling.get("gemiddeld_cijfer", "Niet bekend"))

# Dictionary beheren met een while-loop
database = {}
actief = True

while actief: # Zolang actief gelijk staat aan True (lees meer hierover in 1.7)
    keuze = input("Kies [t]oevoegen, [p]rinten of [s]toppen: ").lower()
    if keuze == "t":
        sleutel = input("Voer sleutel in: ")
        waarde = input("Voer waarde in: ")
        database[sleutel] = waarde
    elif keuze == "p":
        sleutel = input("Welke sleutel wil je inzien? ")
        print("Waarde:", database.get(sleutel, "Niet gevonden"))
    elif keuze == "s":
        actief = False
```

1.7 Loops: for-loops en while-loops

1. Begrensde herhaling (`for` -loop)

Gebruik een `for` -loop als het aantal herhalingen vooraf bekend is, of om direct over elementen van een lijst of tekst te lopen. * `range(stop)` : Loopt van `0` tot `stop` (exclusief). * `range(start, stop[, step])` : Loopt vanaf `start` tot aan `stop` met een instelbare stapgrootte (`1` als je niets invoert).

```
# Loop met range()
for i in range(1, 11):
    print(f"Stap {i}")

# Karakters in een string doorlopen
for letter in "Python":
    print(letter)
```

2. Voorwaardelijke herhaling (`while` -loop)

Blijft draaien zolang de gestelde voorwaarde `True` oplevert. Gebruik deze als vooraf niet vaststaat hoe vaak de code moet herhalen.

```
# Collatz-rij met gehele deling (//)
n = int(input("Startgetal: "))

while n != 1:
    print(n, end=" -> ")
    if n % 2 == 0:
        n = n // 2 # Houdt n een integer (geheel getal)
    else:
        n = 3 * n + 1
print(1)
```

Deel 2: Cybersecurity

2.1 De CIA-Triade

De CIA-triade is het uitgangspunt voor informatiebeveiliging. Een betrouwbaar systeem balanceert alle drie de pijlers:

Pijler	Doel	Concreet risicoscenario bij falen
Vertrouwelijkheid (Confidentiality)	Alleen geautoriseerde personen of systemen kunnen gegevens inzien.	Een slecht beveiligde database lekt medische dossiers of wachtwoorden op internet.
Integriteit (Integrity)	Gegevens blijven aantoonbaar juist, volledig en beschermd tegen onbevoegde wijziging.	Een aanvaller past rekeningnummers of waarden aan in een transactie.
Beschikbaarheid (Availability)	Geautoriseerde gebruikers hebben op elk vereist moment toegang tot systemen en data.	Een webwinkel ligt plat tijdens Black Friday door een grootschalige DDoS-aanval.

Beveiligingsmaatregelen per pijler

- **Maatregelen voor Vertrouwelijkheid:**
 - Versleuteling van data in opslag (*at rest*) en tijdens transport (*in transit* via TLS/HTTPS).
 - Toegangsbeheer via het *Least Privilege*-principe (gebruikers krijgen alleen strikt noodzakelijke rechten).
 - Sterk identiteitsbeheer met wachtwoordbeleid en Multi-Factor Authenticatie (MFA).
- **Maatregelen voor Integriteit:**
 - Cryptografische hashing (zoals SHA-256) en checksums om bestandswijzigingen direct te ontdekken.
 - Digitale handtekeningen om de authenticiteit van software en documenten te garanderen.
 - Versiebeheer, audit logs en strikte validatie van invoervelden tegen injecties.
- **Maatregelen voor Beschikbaarheid:**
 - Redundantie en failover-systemen (dubbele servers, RAID-schijven om hardware-uitval op te vangen).
 - Geteste periodieke back-ups (volgens de 3-2-1 regel: 3 kopieën, 2 verschillende media, 1 externe/offline opslag).
 - DDoS-mitigatie (traffic filtering), load balancers en noodstroomvoorzieningen (UPS).

2.2 Cryptografie en Versleuteling

Cryptografie waarborgt de vertrouwelijkheid en integriteit van data door berichten onleesbaar te maken voor derden.

Eigenschap	Symmetrische Encryptie	Asymmetrische Encryptie
Sleutels	Eén identieke geheime sleutel voor zowel ver- als ontsleuteling.	Een gekoppeld sleutelpaar: publieke sleutel (openbaar, versleutelt) en private sleutel (strikt geheim, ontsleutelt).
Voorbeelden	Caesarcijfer, AES.	RSA, TLS/HTTPS-certificaten.
Sleuteluitwisseling	Kwetsbaar: partijen moeten de geheime sleutel vooraf via een veilig kanaal delen.	Veilig: de publieke sleutel mag openbaar rondgaan; de private sleutel verlaat de eigenaar nooit.
Kraaktechnieken	Brute-force (alle sleutels proberen), frequentieanalyse van letters.	Wiskundige ontbinding van zeer grote getallen in priemfactoren.

```
# Conceptuele Caesar-rotatie in Python (+3 verschuiving)
def caesar_encrypt(tekst, shift=3):
    resultaat = ""
    for char in tekst:
        if char.isalpha():
            basis = ord('A') if char.isupper() else ord('a')
            resultaat += chr((ord(char) - basis + shift) % 26 + basis)
        else:
            resultaat += char
    return resultaat

print(caesar_encrypt("KWIK-INCIDENT")) # NZLN-LQFLGHQW
```

2.3 De Drie Kwetsbaarheden

Beveiligingsrisico's ontstaan in drie lagen:

1. De menselijke factor (gebruikers):

- Zwakke, voorspelbare of hergebruikte wachtwoorden.
- Inspelen op vertrouwen via social engineering (zoals phishing).

2. Communicatie (netwerk en verbindingen):

- Data verzenden via onversleutelde verbindingen (openbare wifi).
- Onderschepping en manipulatie van dataverkeer (*eavesdropping*, MitM).

3. Het systeem (hard- en software):

- Programmeerfouten, buffer overflows en ontwerpfouten in programmacode.
- Verouderde software waarvoor beveiligingsupdates (patches) ontbreken.

2.4 Dreigingsactoren en Attributie

Typen aanvallers

- **Script kiddies:** Onervaren aanvallers die kant-en-klare tools en scripts van anderen draaien, vaak zonder diep technisch inzicht.
- **Cybercriminelen:** Georganiseerde groepen met een financieel motief (ransomware, bankfraude, verkoop van creditcarddata).
- **APT's (Advanced Persistent Threats) & Staatsactoren:** Professionele, zwaar gefinancierde teams verbonden aan inlichtingendiensten. Richten zich op langdurige spionage en sabotage van overheden en vitale infrastructuur.

Wat is attributie?

Attributie is het technisch en juridisch bewijzen wie achter een cyberaanval zit. Dit is uitzonderlijk complex doordat aanvallers: * Verkeer routeren via gekaapte proxy's, VPN's, Tor en botnets. * Doelbewust *false flags* achterlaten (zoals codestukken in een andere taal of tijdzones vervalsen) om andere landen de schuld te geven. * Logbestanden en tussenstations direct wissen na de actie.

2.5 Overzicht Malwaretypen

Type	Werking en Doel
Worm	Schadelijke software die zich geheel zelfstandig over computernetwerken verspreidt zonder dat een gebruiker het bestand hoeft te openen.
Trojan	Vermomt zich als legitieme of handige software om de gebruiker te verleiden het bestand zelf te installeren.
Ransomware	Gijzelsoftware die bestanden of systemen onbreekbaar versleutelt en losgeld eist voor de herstelsleutel.
Rootkit	Nestelt zich diep in het besturingssysteem om beheerdersrechten (<i>root</i>) te behouden en zichzelf voor scanners en virusscanners te verbergen.
Spyware	Verzamelt op de achtergrond privégegevens, surfgedrag en bestanden zonder toestemming van de gebruiker.
Keylogger	Legt elke toetsaanslag exact vast om inloggegevens, chatberichten en betaalgegevens af te vangen.
Adware	Toont ongevraagd pop-ups en advertenties en past vaak standaard zoekmachines in de browser aan.

2.6 Aanvalstechnieken en Stuxnet

Social Engineering & Wachtwoorden (Zwakheid bij de Gebruiker)

- **Phishing:** Massaal verzonden misleidende e-mails of berichten om mensen onder druk te zetten persoonlijke gegevens of geld af te staan.
- **Spearphishing:** Zeer gerichte phishingaanval op één specifieke persoon, student of bedrijfsgroep, verrijkt met persoonlijke details.
- **Whaling:** Spearphishing gericht op directieleden, bestuurders of het hogere management.
- **AI-deepfakefilter:** Het nabootsen van iemands gezicht of stem tijdens een videocall of telefoongesprek om personeel te misleiden.
- **Fysieke social engineering:** Onbevoegd binnendringen door je voor te doen als monteur/ICT-personeel of door een veiligheidsvest aan te trekken.
- **Brute-force-aanval:** Automatisch en systematisch álle mogelijke combinaties van tekens uitproberen tot de juiste sleutel of het wachtwoord gevonden is.
- **Dictionary attack:** Geautomatiseerd inbreken met een *wordlist* (lijsten met miljarden eerder gelekte wachtwoorden zoals `voetball`).
- **Frequentieanalyse:** Kijken naar hoe vaak letters voorkomen in een versleutelde tekst (in het Nederlands vooral **e, n, t**) om een rotatiecijfer zoals Caesar te kraken.

Netwerk- en Communicatieaanvallen (Zwakheid bij de Communicatie)

- **Man-in-the-Middle (MitM):** De aanvaller plaatst zich ongemerkt tussen twee communicerende partijen om netwerkverkeer af te luisteren of aan te passen.
- **Evil Twin:** Een kwaadaardig wifi-toegangspunt (bijv. een mobiele hotspot) met exact dezelfde naam als een legitiem gastnetwerk om verkeer en inloggegevens te onderscheppen.
- **DNS-spoofing:** De gegevens van een DNS-server manipuleren zodat webadressen (zoals van een bank) stiekem doorverwijzen naar het IP-adres van een identiek nagemaakte server van de hacker om op de nepsite inloggegevens te stelen.
- **DDoS-aanval via botnet:** Een server of website overbelasten en laten crashen door miljoenen gelijktijdige verzoeken te sturen vanuit een netwerk van gekaapte apparaten (van pc's tot IoT-apparaten zoals routers en camera's).
- **Stingray:** Een valse mobiele zendmast die al het mobiele data- en belverkeer van telefoons in de omgeving fysiek onderschept.
- **Typosquatting:** Het registreren van een domeinnaam die lijkt op een bestaande website maar een typfout bevat (bijv. `gogle.com`) om bezoekers te lokken naar een criminele site of om malware te verspreiden.

Systeemexploitatie (Zwakheid in het Systeem)

- **RCE (Remote Code Execution):** Een kwetsbaarheid waarmee een aanvaller op afstand willekeurige programmacode of commando's uitvoert op een ander systeem.
- **Root Access:** Het verkrijgen van de allerhoogste administratieve beheerdersrechten over het systeem (vaak het vervolg op een RCE).
- **Living-off-the-land:** Een techniek waarbij aanvallers geen eigen malware uploaden, maar slim gebruikmaken van tools en software die al standaard op het doelsysteem aanwezig zijn (zoals netwerkscanners) om detectie te omzeilen.
- **SQL-injectie (SQLi):** Database-commando's via een invoerveld injecteren om authenticatie te omzeilen of de database uit te lezen.

```
-- Voorbeeld van kwetsbare SQL-query bij foute invoer: ' OR '1'='1
SELECT * FROM data WHERE name == 'admin' AND passwd == 'welkom123';
-- Geïnjecteerd met: '; SELECT * FROM data; --
SELECT * FROM data WHERE name == ''; SELECT * FROM data; --
```

- **Zero-day exploit:** Een aanval die misbruik maakt van een fout in software die nog niet bekend is bij de ontwikkelaar, waardoor er nul dagen de tijd is geweest om een update of patch uit te brengen.

Stuxnet

Stuxnet (ontdekt rond 2010) was het eerste grootschalig ingezette cyberwapen. Het werd ontwikkeld door statelijke actoren om het Iraanse kernprogramma in Natanz te saboteren.

- **Werkwijze:** Het doelsysteem was volledig afgesloten van internet (*air-gapped*). Stuxnet overbrugde dit via geïnfecteerde USB-sticks.
- **Exploits:** De worm gebruikte vier verschillende zero-day-kwetsbaarheden en digitale handtekeningen van legitieme bedrijven.
- **Doel:** Specifieke frequentieomvormers van centrifuges via PLC-controllers ontregelen. Stuxnet liet centrifuges wisselend overtoeren draaien en plotseling afremmen, waardoor ze fysiek kapotgingen. Tegelijkertijd stuurde een ingebouwde rootkit normale meetwaarden naar de controlemonitoren, zodat technici maandenlang dachten aan mechanische pech.

2.7 Beveiligingsmaatregelen en Verdedigingsstrategieën

Toegangsbeheer

- **Wachtwoordmanagers:** Genereren en versleutelen unieke, sterke wachtwoorden per dienst.
- **Passphrases:** Reeksen willekeurige woorden (*kat-fiets-maan-koffie*) die door hun lengte praktisch bestand zijn tegen brute-force-aanvallen en makkelijker te onthouden zijn.
- **MFA (Multi-Factor Authenticatie):** Vereist minstens twee bewijsstukken uit verschillende categorieën:
 1. *Wat je weet* (wachtwoord/pincode).
 2. *Wat je hebt* (authenticator-app, sms-token, fysieke YubiKey).
 3. *Wat je bent* (biometrie: vingerafdruk of gezichtsscan).

Netwerk en Analyse

- **Firewall:** Inspecteert inkomend en uitgaand netwerkverkeer en blokkeert verdachte pakketten op basis van regels en poortnummers.
- **OSINT (Open Source Intelligence):** Kwetsbaarheden of datalekken opsporen via legaal toegankelijke openbare bronnen.
- **VirusTotal:** Online multiscanner die geüploade bestanden en links tegelijkertijd controleert met ruim 70 checks.

Beleid en Ethisch Hacken

- **Meldplicht Datalekken:** Wettelijke verplichting (onder de AVG) om incidenten met persoonsgegevens binnen 72 uur te melden aan de Autoriteit Persoonsgegevens.
- **Ethisch hacken & Responsible Disclosure:** Kwetsbaarheden zoeken met voorafgaande schriftelijke toestemming en deze discreet melden aan de eigenaar vóór openbaarmaking.
- **Red Team vs. Blue Team:**
 - *Red Team:* Simuleert de rol van aanvallers om de effectiviteit van de verdediging te beproeven.
 - *Blue Team:* Monitort de systemen, detecteert incidenten en verdedigt de infrastructuur actief tegen aanvallen.

Deel 3: Digitale Privacy en Maatschappij

3.1 Trackingtechnologieën en Digitale Profilerings

- **Cookies:**
 - *First-party cookies:* Geplaatst door het bezochte domein om basisfuncties (zoals inlogstatus of winkelmandje) te bewaren.
 - *Third-party cookies:* Geplaatst door externe advertentienetwerken om surfgedrag over meerdere websites te koppelen aan een profiel.
- **Browser Fingerprinting:** Een volgmethode die tientallen technische systeemeigenschappen (schermresolutie, geïnstalleerde lettertypen, GPU-canvas, browserplugins, tijdzone) combineert tot een unieke identifier, zonder afhankelijk te zijn van cookies.
- **Digitale profilering:** Gedragsdata grootschalig analyseren met machine-learningmodellen om voorkeuren, emoties en toekomstig koop- of stemgedrag te voorspellen.
- **Dark Patterns:** Manipulatieve interface-ontwerpen die gebruikers sturen naar privacy-onvriendelijke keuzes (bijv. een felgekleurde 'Accepteer alles'-knop naast een verborgen linkje voor cookie-instellingen).

3.2 Maatschappelijke Casussen

De Toeslagenaffaire

- **Oorzaak:** De Belastingdienst zette geautomatiseerde risico-algoritmes in om fraude met kinderopvangtoeslag op te sporen.
- **Mechanisme:** De gebruikte modellen bevatten vooringenomenheid (*algorithmic bias*): kenmerken zoals een dubbele nationaliteit kregen automatisch een hogere risicoscore.
- **Gevolgen:** Tienduizenden burgers werden zonder deugdelijk bewijs als fraudeur bestempeld, toeslagen werden acuut stopgezet en onterecht teruggevorderd. Dit veroorzaakte enorme financiële en persoonlijke schade en leidde tot de val van het toenmalige kabinet.

Cambridge Analytica

- **Oorzaak:** Een schijnbaar onschuldige persoonlijkheidstest via Facebook verzamelde niet alleen de data van testdeelnemers, maar ook die van al hun vrienden.
- **Mechanisme:** Het databedrijf Cambridge Analytica verkreeg zo ongeoorloofd data van tientallen miljoenen kiezers en koppelde dit aan psychografische profielen.
- **Gevolgen:** Gerichte politieke advertenties (*micro-targeting*) werden ingezet om kiezersgedrag te manipuleren tijdens de Amerikaanse presidentsverkiezingen van 2016 en het Brexit-referendum. Dit versnelde de roep om strenge privacywetgeving wereldwijd.

3.3 Wet- en Regelgeving

- **AVG / GDPR (Algemene Verordening Gegevensbescherming):**

Europese privacywet die strenge eisen stelt aan het verzamelen, opslaan en verwerken van persoonsgegevens. Geeft burgers rechten zoals inzage, correctie en het recht op vergetelheid.

- **Autoriteit Persoonsgegevens (AP):**

De Nederlandse toezichthouder die controleert op naleving van de AVG en bevoegd is om dwangsommen en boetes (tot 20 miljoen euro of 4% van de wereldwijde jaaromzet) op te leggen.

- **DSA (Digital Services Act) & DMA (Digital Markets Act):**

Europese wetgeving die grote techplatformen ("gatekeepers") dwingt om transparant te zijn over aanbevelingsalgoritmes, illegale content actiever te weren en eerlijke concurrentie mogelijk te maken.

3.4 Privacy Enhancing Technologies (PETs)

- **VPN (Virtual Private Network):**

Bouwt een versleutelde tunnel tussen het apparaat en de VPN-server. Beschermde data tegen af luisteren op lokale netwerken en verbergt je echte IP-adres voor de bezochte website.

- **End-to-End Encryptie (E2EE):**

Bij end-to-end encryptie worden berichten versleuteld op het apparaat van de afzender en pas ontsleuteld op het apparaat van de ontvanger. Tussentijdse servers, internetproviders en de app-ontwikkelaar kunnen de inhoud niet inzien.